

Chiffrer la connexion entre le navigateur et le serveur web

Note : Pour pouvoir faire ce TP, il faut avoir fait les TP « héberger une page web », « enregistrer un nom de domaine pour votre conteneur » et « servir plusieurs sites web avec le même serveur ». La configuration de virtualhosts dédiés pour chacun de vos domaines est nécessaire.

L'objectif de ce TP est de permettre le chiffrement de la connexion entre le serveur web installé sur votre conteneur et les clients qui vont s'y connecter.

Pourquoi chiffrer ?

Le but de cette introduction est de comprendre la nécessité de chiffrer. Vous pouvez l'effectuer sur votre système personnel.

1. Installez le paquet **ngrep**, regardez rapidement la page du manuel de la commande **ngrep**.
2. Dans un terminal, exécutez la commande suivante qui va afficher le contenu de tous les segments qui ont 80 comme port source ou destination.

```
# ngrep -W byline port 80
```

3. Depuis un navigateur web, consultez la page web <http://formtest.netlib.re/> et renseignez un login et un mot de passe de votre choix. Si votre navigateur web insiste pour transformer l'URL <http://formtest.netlib.re/> en <https://formtest.netlib.re/> vous pouvez utiliser le navigateur web en mode texte **elinks** ou le navigateur léger **dillo**.
4. Que constatez vous ?
5. Refaites la même opération, mais en observant le port 443 et en remplaçant **http** par **https** dans l'URL de la page web.
6. Que constatez vous ?

Ainsi, un message qui circule en clair peut facilement être observé par un attaquant passif qui pourrait obtenir des données personnelles.

Le retour du MITM !

Comme pour SSH, afin d'établir une connexion chiffrée, le client (C) et le serveur (S) doivent se mettre d'accord sur une clef de chiffrement commune. Comme pour SSH, si un attaquant (A) se place entre le client C et le serveur S, il peut se faire passer pour le serveur S auprès du client C (et se mettre d'accord avec C sur une clef de chiffrement dont il a le contrôle) et se faire passer pour le client C auprès du serveur S (et se mettre d'accord avec S sur une autre clef de chiffrement dont il a le contrôle).

Si le serveur S est le seul à posséder une clef privée et que le client C possède la clef publique correspondante, alors le serveur S peut s'authentifier auprès du client C (et l'attaquant A ne peut pas se faire passer pour le serveur S auprès du client C lors de l'établissement de la clef de chiffrement commune).

7. Relisez le TP « Première connexion SSH ».

Comment transmettre une clef publique ?

Reste le problème suivant : comment le serveur S peut-il transmettre la clef publique correspondant à sa clef privée au client C ?

Le serveur S ne peut pas simplement envoyer sa clef publique au client C car un attaquant A qui se trouverait sur la route pourrait la changer et se faire passer pour le serveur S auprès du client C.

Un mécanisme permettant cette transmission s'appelle une infrastructure à clef publique (en: public-key infrastructure, PKI).

Dans le cas de SSH, la transmission de la clef publique du serveur se fait de la main à la main, si possible lors d'un contact physique. En effet, une personne qui administre un système connaît en général les personnes qui vont s'y connecter.

Lorsque vous avez invité vos collègues sur votre conteneur, vous avez vérifié ensemble que le fingerprint qu'affichait le client SSH de vos invité-es correspondait bien au fingerprint d'une des clefs publiques de votre serveur SSH et vous avez fait cette vérification en présence. Ainsi, vos collègues avaient une entière confiance au fait que la clef publique fournie par votre serveur SSH était la bonne (et pas une fausse clef créée par un-e attaquant-e se trouvant sur la route entre votre conteneur et le portable de vos collègues).

Le problème pour un serveur web est qu'on ne connaît pas à l'avance les personnes qui vont s'y connecter. Ainsi, il n'est pas pensable de vérifier en présence la clef publique d'un serveur web avant de pouvoir s'y connecter.

Le protocole HTTPS n'est autre que HTTP over TLS (successeur de SSL). Autrement dit, les serveurs et clients web, délèguent la gestion de l'authentification et du chiffrement à TLS.

La solution proposée par TLS est l'utilisation d'une autorité de certification : le client web possède une liste de clefs publiques d'organismes habilités à signer les clefs publiques des serveurs web. Le serveur web du nom de domaine `example.netlib.re` fait signer sa clef publique à une ces autorités en prouvant qu'il détient bien le nom de domaine `example.netlib.re` (soit en répondant à des challenges, soit en envoyant une preuve de paiement du nom de domaine, etc). Lorsque le client web reçoit la clef publique du serveur web, il reçoit aussi cette signature (appelée certificat) et peut vérifier que la clef publique du serveur web a bien été signée par la clef privée de l'autorité de certification.

Remarque : souvent les clefs publiques des organismes de certifications sont possédées par le système d'exploitation et partagées aux clients web (`wget`, `firefox`, `chromium`, etc). Sous Debian, le répertoire `/etc/ssl/certs/` contient des liens symboliques vers ces clefs (SSL est l'ancêtre de TLS, le nom du répertoire a été préservé).

8. Faites un dessin de la situation en mettant en jeu le serveur HTTP, des clients HTTP, des attaquants, l'autorité de certification, les diverses clefs (publiques ou privées) et les certificats.
9. Tant que vous y êtes, lisez la feuille de debriefing sur les notions d'identification, authentification, autorisation.

Remarque importante

La paire de clefs publique/privée du serveur sert à l'*authentification*. La clef de chiffrement utilisée pour le *chiffrement* de la connexion entre le client et le serveur n'a rien à voir.

Dans le premier cas, il s'agit de *cryptographie asymétrique*, dans le second cas, il s'agit de *cryptographie symétrique*. Les algorithmes utilisés sont très différents.

Le protocole d'établissement d'une clef de chiffrement commune (une fois l'authentification du serveur par le client effectuée) repose actuellement sur la méthode de Diffie-Hellman

Toutes ces notions seront approfondies dans le cours de cryptographie en L3.

Let's Encrypt

Jusqu'à récemment, l'obtention d'un certificat auprès d'un organisme de certification était compliquée et chère.

Nous allons utiliser l'autorité de certification Let's Encrypt, qui est reconnue par la plupart des systèmes d'exploitation et/ou navigateurs.

Cette autorité de certification a été fondée par l' Electronic Frontier Foundation, une ONG de défense des libertés sur internet, la fondation Mozilla (qui développe entre autres le navigateur firefox) et l'université du Michigan afin de généraliser l'utilisation du chiffrement sur internet (en particulier l'utilisation de HTTPS), en simplifiant radicalement l'étape de certification (via des challenges automatisés entre Let's Encrypt et le serveur web) et en la rendant gratuite.

Dans la pratique

Le logiciel **certbot** permet de créer une paire de clefs et de la faire certifier par Let's Encrypt (en répondant aux challenges) et même de modifier la configuration de vos virtualhosts.

10. Cherchez le paquet **certbot** qui corresponde à votre serveur web (pensez à combiner les commandes **apt search** et **grep**).
11. Consultez la description du paquet à l'aide d'une sous-commande d'**apt** (**man apt**).
12. Installez le paquet.
13. Sauvegardez le fichier de configuration du virtualhost correspondant au nom de domaine que vous voulez certifier (par exemple copiez `/etc/nginx/sites-available/<vhost>` vers `/etc/nginx/sites-available/<vhost>.bak`).
14. Créez une paire de clefs pour un site installé sur votre serveur (et répondant à un nom de domaine particulier) et faites la certifier par Let's Encrypt, le tout en un seul appel à la commande **certbot** (consultez les options **-d** (ou **--domain**) et **--nginx**).
15. Observez comment le fichier de configuration du virtualhost correspondant à votre nom de domaine a changé (vous pouvez par exemple utiliser la commande **diff** ou **vimdiff** pour voir les différences côte à côte).
16. Observez le répertoire `/etc/cron.d/`, quelle est sa fonction ? Observez le fichier **certbot** qui s'y trouve.
17. Allez sur <https://www.ssllabs.com/ssltest> et testez la qualité des configurations mises en place par **certbot**. Vous devriez avoir la note **A**.
18. Lorsque vous avez obtenu un certificat TLS et que les clients de votre serveur web peuvent s'y connecter en HTTPS, vous pouvez ajouter **web.tls** à vos tags.
19. Identifiez l'item qui ne va pas (en orange), et consultez la page de blog liée.

Les failles du protocole X.509

Le protocole X.509 de certification par autorité de certification a le problème majeur suivant : toutes les autorités de certification reconnues par le client web peuvent signer des clefs publiques concernant votre nom de domaine. Il est arrivé, à plusieurs reprises que des autorités de certifications (commerciales ou étatiques) signent de mauvaises clefs (pour des raisons financières ou de surveillance par exemple).

En général, ces fraudes finissent par se voir et l'autorité frauduleuse finit par être exclue des listes installées dans les navigateurs.

Ainsi, ça n'est pas parce que vous avez utilisé une autorité de certification fiable pour certifier votre domaine que les communications entre votre serveur web et ses clients le sont.

Il suffit d'une mauvaise autorité de certification dans la liste du client pour qu'une attaque MITM soit possible.

Il suffit aussi qu'un attaquant arrive à se faire passer par vous auprès d'une autorité de certification de bonne foi pour qu'une attaque MITM soit possible.

Le DNS CAA sert à empêcher cette dernière attaque (mais pas la précédente).

La RFC 6844 introduit l'enregistrement **CAA** dans le DNS qui permet à l'entité qui possède un nom de domaine de spécifier quels autorités de certifications peuvent certifier ce domaine. Ainsi, si un attaquant réussit à se faire passer pour le propriétaire d'un nom de domaine, il ne pourra pas se faire certifier par n'importe quelle autorité, car celle-ci refusera d'émettre un certificat si elle voit que ce champ existe et ne lui correspond pas.

20. Faites en sorte que seule l'autorité **Let's Encrypt** accepte de certifier votre nom de domaine.
21. Vérifiez auprès de **ssllabs** que votre modification fonctionne bien.
22. Lorsque vous avez mis en place un enregistrement **CAA** et que seule l'autorité **Let's Encrypt** accepte de certifier votre nom de domaine, vous pouvez ajouter **web.caa** à vos tags.